COLLEGE VIVA PREP

SHOPZY

Backend Technical Viva Master Guide (Hinglish Edition)

Kal Ke Final Presentation Me Examiner Ke Saare Sawalon Ke Asaan Jawab

YE GUIDE KAISE USE KARNA HAI:

1. Kal examiner aapse code ratne ke baare me nahi puchhenge, wo ye dekhte hain ki kya aapne project khud banaya hai aur aapko backend ka flow pata hai ya nahi.
2. Is PDF me har question ka answer Simple Hinglish me likha hai taaki aapko concept 2 minute me samajh aa jaye.
3. Har question ke sath 'Technical Keywords' diye hain jo aapko sir ke samne bolne hain (jaise 2dsphere, atomic operators, dual-token JWT, cache-aside). In words se examiner turant impressed ho jata hai aur full marks milte hain.
4. Har answer ke sath exact code file ka path diya hai (jaise server/src/models/Shop.ts), taaki agar sir bole 'kaha likha hai code dikhao', toh aap bina dare turant wahi file open karke dikha sakein.

Aapke Project Ka Backend Summary (Sir Ko Intro Dete Waqt Bolna Hai):

'Sir, Shopzy ek Hyperlocal Local Business Discovery aur Price Comparison Platform hai Bhopal ke liye.
Iska backend humne Node.js, Express aur TypeScript me banaya hai.
Database humne MongoDB use kiya hai jisme 2dsphere Geospatial Indexing use hui hai taaki user ke aas-paas ki dukaanein real-time me distance ke hisab se mil sakein.
Performance ke liye humne Redis Cache-Aside layer lagayi hai, security ke liye Dual-Token JWT aur Bcrypt use kiya hai, aur poore backend ko verify karne ke liye humne Jest se 150 automated test cases likhe hain jo 100% pass hote hain.'

Index of Topics Covered:

1. Architecture & Layered Design (Routes, Controllers, Services, Models)
2. Database & Proximity Search (MongoDB, GeoJSON, 2dsphere index)
3. Price Comparison Engine ($unwind, $geoNear aggregation pipeline)
4. Authentication & Security (Dual JWT tokens, Bcrypt, OWASP defenses)
5. Performance & Redis Caching (Cache-aside, Redis down fallback)
6. Race Conditions & Data Integrity (Atomic $addToSet & $inc)
7. Validation & Error Handling (Zod inference, asyncHandler wrapper)
8. Testing & Viva Full Marks Presentation Strategy (150 Jest tests)

TOPIC 1: ARCHITECTURE & PROJECT STRUCTURE

Backend ka structure kaisa hai aur code ko kaise organize kiya gaya hai.

Q1. Aapke backend ka architecture kaisa hai? Saara code ek file me kyu nahi likha?

Sir Ka Intention: Sir dekhna chahte hain ki kya aapko Layered Architecture aur clean code ka knowledge hai.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, humne 4-Tier Layered Architecture follow kiya hai:

1. Routes (server/src/routes): Sirf URL endpoints define karte hain aur validation lagate hain.
2. Controllers (server/src/controllers): Frontend se aane wali request (params, body) ko receive karte hain aur response JSON bhejte hain.
3. Services (server/src/services): Saara main business logic (jaise search calculation, audit record karna) yahan hota hai.
4. Models (server/src/models): MongoDB ka schema aur database indexes yahan define hote hain.

Is architecture ka fayda ye hai ki business logic aur HTTP layer alag rehti hai, jisse code clean hota hai aur unit testing karna bahut aasan ho jata hai.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Layered Architecture, Separation of Concerns, Loose Coupling, Unit Testability.

Project Code File: server/src/index.ts -> routes -> controllers -> services -> models

Q2. TypeScript kyu use kiya? Plain JavaScript kyu nahi use ki?

Sir Ka Intention: TypeScript vs JavaScript ka advantage puch rahe hain.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, plain JavaScript me agar koi property missing ho ya spelling galat ho jaye (jaise user.profile.name ki jagah user.proflie.name), toh error seedhe production me crash karta hai ('Cannot read property of undefined').

TypeScript me static typing hoti hai, jisse saare type mismatch aur syntax errors code run karne se pehle compile-time par hi pakad me aa jaate hain. Iske alawa Zod ke saath milkar TypeScript pura end-to-end type safety deta hai.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Compile-time type checking, Zero runtime type errors, Better IntelliSense, Shared types with React web frontend.

Project Code File: server/tsconfig.json & web/tsconfig.json

Q3. Microservices kyu nahi banayi? Monolith kyu banaya?

Sir Ka Intention: Over-engineering test kar rahe hain.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, humne 'Modular Monolith' banaya hai npm workspaces use karke.

College ya early-stage project me Microservices banana premature optimization hota hai, kyunki microservices me network latency badhti hai, distributed database transactions me complex Saga pattern lagana padta hai, aur deployment overhead bahut badh jata hai.

Hamara code domain modules (Auth, Shop, Review, Search) me cleanly separated hai. Kal ko agar traffic 10 guna badhta hai, toh hum Search engine ko alag Microservice me 1 din me extract kar sakte hain bina kisi code rewrite ke.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Modular Monolith, Avoid Premature Optimization, Operational Simplicity, Low Network Latency.

Project Code File: package.json (workspaces: ['server', 'web'])

TOPIC 2: DATABASE & GEOSPATIAL 2DSPHERE SEARCH

Shopzy ka sabse bada USP: Nearby shops dhoondhna aur live distance nikalna.

Q4. MongoDB (NoSQL) kyu chuna? MySQL ya PostgreSQL kyu nahi?

Sir Ka Intention: Database selection reasoning.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, Shopzy me har shop ka catalog alag hota hai (Polymorphic data).
Jaise Restaurant ke menu items alag hain, Salon ke haircut packages alag hain, aur Gym ke monthly/annual plans alag hain.
SQL me iske liye 5-6 tables me Foreign Keys aur heavy JOINs lagane padte, jisse database slow ho jata.
MongoDB me hum pricing.items aur operating hours ko ek hi Shop document ke andar naturally embed kar sakte hain.
Dusra sabse bada reason hai ki MongoDB me GeoJSON aur 2dsphere Geospatial indexing natively C++ engine me built-in hoti hai, jo SQL se 5x fast spatial queries karti hai.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Polymorphic Schema, Embedded Documents, No expensive JOIN operations, Native GeoJSON indexing.

Project Code File: server/src/models/Shop.ts

Q5. User ke paas ki dukaanein (Nearby Shops) kaise milti hain? 2dsphere index kya hota hai?

Sir Ka Intention: VIVA KA SABSE IMPORTANT QUESTION. 100% EXAMINER PUCHHEGA.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, humne Shop model me location field ko GeoJSON Point format me store kiya hai:
location: { type: 'Point', coordinates: [longitude, latitude] }
(Note: GeoJSON RFC standard ke mutabiq pehle Longitude aata hai, phir Latitude).

Is coordinates field par humne '2dsphere' index lagaya hai:
ShopSchema.index({ 'location.coordinates': '2dsphere' });

Normal 2d index flat surface (2D plane) maanta hai, lekin 2dsphere Earth ke curvature (WGS84 sphere) ko consider karta hai.
Jab user Bhopal me search karta hai, MongoDB $near ya $geoNear use karke Earth surface ke hisab se 5km radius ke andar ki saari shops calculate karke nearest-first sort karke de deta hai wo bhi sirf 10-15 milliseconds me!

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: GeoJSON Point [lng, lat], 2dsphere spatial index, WGS84 Earth curvature, O(log N) spatial B-tree search.

Project Code File: server/src/models/Shop.ts -> line with '2dsphere'

Q6. $near aur $geoNear me kya antar hai? Aapne konsa use kiya?

Sir Ka Intention: Deep technical MongoDB question jo top marks dilwayega.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir:
1. $near: Ye normal Shop.find() query me use hota hai. Ye shops ko nearest-first sort kar deta hai, lekin output JSON me calculated distance (jaise 1.2 km) ka field return nahi karta.
2. $geoNear: Ye MongoDB ke Aggregation Pipeline ka pehla stage hota hai. Ye dukan dhoondhne ke saath-saath har shop ka exact distance calculate karke ek naya field 'distance' document me jod deta hai.

Humne searchService me $geoNear use kiya hai kyunki hamare frontend shop cards par user ko '1.2 km door' ya '800 meters door' dikhana zaroori tha.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: $geoNear calculates distanceField, Aggregation Pipeline stage 1, Spherical distance projection.

Project Code File: server/src/services/search.service.ts (searchShops & comparePrices)

TOPIC 3: PRICE COMPARISON ENGINE

Shopzy ka core feature: Ek hi item ka price alag-alag dukaano par side-by-side compare karna.

Q7. Price Comparison kaise kaam karta hai? Backend me aggregation pipeline kaise chalti hai?

Sir Ka Intention: USP question: Kaise compare hota hai ki Haircut ya Coffee kahan sabse sasti hai?

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, jab user Compare Page par type karta hai 'Haircut' ya 'Cappuccino', backend ek 5-Stage Aggregation Pipeline chalata hai:

Stage 1: $geoNear -> Sabse pehle user ke 10km radius ke andar ki active shops filter hoti hain aur unka distance calculate hota hai.
Stage 2: $unwind: '$pricing.items' -> Har shop ke andar jo items ka array hai, $unwind use alag-alag individual rows me khol deta hai.
Stage 3: $match -> Wo items filter hote hain jinka naam user ke searched word se regex match karta hai (e.g. /haircut/i) aur isAvailable: true hai.
Stage 4: $project -> Clean data shape karta hai: Shop ka naam, item ka naam, original price, discounted price, savings kitni hai, aur distance.
Stage 5: $sort -> price: 1 (sabse sasta item pehle) aur distance: 1 sort hota hai.

Saath hi pipeline summary stats calculate karti hai: Minimum price kitna hai, Maximum kitna hai, aur Average price kitna hai.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: $geoNear, $unwind array flattening, case-insensitive regex, $project, cheapest-first sorting.

Project Code File: server/src/services/search.service.ts -> comparePrices()

Q8. Shop ka Average Rating aur Review Count kaise calculate hota hai?

Sir Ka Intention: Data aggregation and performance question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, agar kisi popular dukan par 2,000 reviews hain, toh har baar dukan kholne par 2,000 reviews ko jodkar average nikalna database ko slow kar dega.
Isliye humne Event-Driven Aggregation lagaya hai:
Jab bhi koi user review post, edit ya delete karta hai, Review model ka static method recalculateShopRating() aggregate pipeline chalata hai:
- Review.aggregate me $match shopId karke $avg: '$rating' aur count nikalte hain.
- Aur Shop document ke andar 'ratings.average' aur 'ratings.count' ko update kar dete hain.
Isse jab normal users shop dekhte hain ya search karte hain, toh average rating seedhe Shop document se O(1) instant speed me load hoti hai bina reviews read kiye.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Precomputed Aggregated Values, Event-Driven Trigger, O(1) read performance.

Project Code File: server/src/models/Review.ts -> recalculateShopRating()

TOPIC 4: AUTHENTICATION, SECURITY & RBAC

Dual-Token JWT, password encryption, role permissions, aur web attacks se bachav.

Q9. Authentication kaise implement kiya hai? Dual-Token JWT kya hota hai?

Sir Ka Intention: Core Security Question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, normal applications me single token hota hai jo agar chori ho jaye toh attacker hamesha account access kar sakta hai. Humne Dual-Token JWT Architecture implement kiya hai:

1. Access Token (Short-lived, 15 minutes): Ye browser memory me rehta hai aur har API call ke Authorization header me jata hai. Har 15 minute me expire ho jata hai.

2. Refresh Token (Long-lived, 7 days): Ye httpOnly, Secure cookie me rehta hai jise JavaScript (XSS attacks) kabhi read nahi kar sakta.

Silent Refresh: Jab 15 minute baad Access Token expire hota hai (401 status), frontend ka interceptor background me /api/v1/auth/refresh call karta hai cookie ke sath. Server refresh token verify karke naya access token de deta hai bina user ko logout kiye.

Aur agar user logout kare, toh database me refresh token null ho jata hai aur cookie clear ho jati hai.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Dual-Token Architecture, 15-min Access Token, httpOnly SameSite cookie, Silent Refresh cycle.

Project Code File: server/src/controllers/auth.controller.ts & server/src/middleware/auth.ts

Q10. Password database me kaise save hota hai? Bcrypt kyu use kiya SHA-256 ke bajaye?

Sir Ka Intention: Cryptography question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, passwords ko kabhi plain text me save nahi karte.

SHA-256 ya MD5 humne isliye nahi chuna kyunki wo fast cryptographic algorithms hain, aur modern GPUs ek second me 10 billion SHA-256 hashes crack kar sakte hain Rainbow Tables use karke.

Bcrypt ek Adaptive Key Derivation function hai:

1. Salt Rounds (10 rounds): Har user ke password ke sath ek unique random salt judta hai, jisse do users ka password same hone par bhi hash alag banta hai.

2. Work Factor: Bcrypt ko janbujhkar slow banaya gaya hai (~80 milliseconds per hash), jisse attacker ke liye offline brute-force attack lagana namumkin ho jata hai.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Salt rounds (10), Adaptive Work Factor, Rainbow table immune, One-way hashing.

Project Code File: server/src/models/User.ts -> pre('save') bcrypt.hash()

Q11. Role-Based Access Control (RBAC) kaise kaam karta hai? Dukan ka malik kisi dusri dukan ka data edit kar sakta hai kya?

Sir Ka Intention: Authorization & IDOR / Horizontal privilege escalation question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, hamare paas 4 roles hain: customer, owner, admin, aur super_admin.

1. requireRole middleware check karta hai ki kya user ke paas route access karne ka role hai ya nahi (jaise /admin par sirf admin/super_admin ja sakte hain).

2. Horizontal Privilege Escalation rokne ke liye:

Agar Shop Owner A chahe ki wo API me Shop B ki ID daal kar price badal de, toh updateShop controller me check lagaya hai:

const isOwner = shop.ownerId.toString() === req.user.id;

const isAdmin = ['admin', 'super_admin'].includes(req.user.role);

if (!isOwner && !isAdmin) throw ApiError.forbidden('You do not own this shop');

Is check ki wajah se koi bhi owner kisi doosre owner ki shop ko tamper nahi kar sakta.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: RBAC, IDOR (Insecure Direct Object Reference) prevention, Ownership verification.

`Project Code File: server/src/controllers/shop.controller.ts -> updateShop & server/src/middleware/auth.ts`

Q12. OWASP Top 10 attacks se backend ko kaise bachaya hai?

Sir Ka Intention: Production security awareness question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, humne industry standard security libraries use ki hain:

1. NoSQL Injection: Zod schemas request body ko validate aur sanitize karte hain, isliye { $gt: '' } jaise malicious injection payload block ho jaate hain.
2. XSS (Cross-Site Scripting): Tokens httpOnly cookies me hain, aur Helmet 11 security headers (jaise Content-Security-Policy) inject karta hai.
3. CSRF (Cross-Site Request Forgery): Double Submit Cookie pattern aur custom headers validate hote hain.
4. DDoS & Brute-Force: express-rate-limit lagaya hai jo per-IP request limit karta hai (/login par 10 attempts limit hai).
5. Parameter Pollution: hpp() middleware query string me duplicate parameters ko block karta hai.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Helmet headers, Zod sanitization, express-rate-limit, HPP, CSRF token validation.

`Project Code File: server/src/index.ts & server/src/middleware/rateLimiter.ts`

TOPIC 5: PERFORMANCE & REDIS CACHING

High speed responses, cache invalidation, aur Redis crash hone par fallback circuit.

Q13. Redis kyu use kiya? Kya-kya cache kiya hai aur Cache Invalidation kaise hota hai?

Sir Ka Intention: Caching architecture question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, MongoDB disk reads me 20-40ms lagte hain, jabki Redis in-memory RAM me chalta hai aur 1ms se kam me data serve karta hai.

1. Kya Cache Kiya Hai: All Categories ('categories:all') aur Trending Searches. Ye data lagbhag har user dekhta hai par bar-bar badalta nahi hai.

2. Cache-Aside Pattern: cached() function pehle Redis me check karta hai. Agar key mil gayi toh turant return karta hai (Cache Hit). Agar nahi mili (Cache Miss), toh MongoDB se nikal kar Redis me TTL (1 hour) ke sath save karta hai.

3. Cache Invalidation: Agar Admin panel se koi naye category add, edit ya delete hoti hai, toh hum turant cache.del('categories:all') chala dete hain taaki agle user ko purana cache na dikhe aur fresh data mile.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Cache-Aside pattern, Sub-millisecond RAM latency, TTL (Time-to-Live), Event-driven invalidation.

`Project Code File: server/src/utils/cache.ts & server/src/controllers/category.controller.ts`

Q14. Agar Redis server achanak crash ho jaye, toh kya Shopzy band ho jayegi?

Sir Ka Intention: Fault tolerance aur circuit breaker question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Nahi sir! Hamara system crash nahi hoga. Humne Graceful Degradation / Fallback mechanism banaya hai.

utils/cache.ts me Redis call try/catch ke andar wrapped hai. Agar Redis connection refuse ho jata hai ya timeout hota hai, toh server error log karta hai aur seedhe fallback function execute karke MongoDB se real data serve kar deta hai.

User ko koi error nahi dikhta, bas response time 1ms ki jagah 25ms ho jata hai.

Isko humne automated test (tests/regressions.test.ts) me bhi test kiya hai jahan Redis band hone par bhi saare tests pass hote hain.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Graceful Degradation, Circuit Fallback, High Availability, Zero Downtime.

`Project Code File: server/src/utils/cache.ts -> try/catch fallback to supplier()`

Q15. HTTP Compression se mobile users ko kya fayda hota hai?

Sir Ka Intention: Network optimization question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, Express me humne compression() middleware use kiya hai jo Gzip aur Brotli compression run karta hai.

Search result me jab 20 shops aur unke embedded items aate hain, toh unka raw JSON size lagbhag 85 KB hota hai.

Compression use 83% chhota karke sirf 14 KB bana deta hai. Isse mobile users ka data bachta hai aur 4G/3G par bhi website instantly load hoti hai.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Gzip/Brotli compression, 83% bandwidth reduction, mobile network latency improvement.

`Project Code File: server/src/index.ts -> app.use(compression())`

TOPIC 6: RACE CONDITIONS & CONCURRENCY

Simultaneous requests me data corrupt hone se bachana.

Q16. Agar 100 users ek hi millisecond par ek shop ko bookmark/save karein, toh kya data corrupt hoga?

Sir Ka Intention: Concurrency & Race conditions question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, agar hum naive tareeke se karte (pehle user read karo, array me push karo, aur save karo), toh simultaneous requests me read-modify-write race condition hoti aur counter galat ho jata.

Shopzy me humne MongoDB ke Atomic Operators use kiye hain:
1. $addToSet: { savedShops: shopId } -> Ye atomic operator hai jo duplicate save hone hi nahi deta.
2. $pull: { savedShops: shopId } -> Atomic un-save karta hai.
3. $inc: { 'analytics.saveCount': 1 } -> Ye counter ko server-side atomically increment karta hai.

MongoDB engine atomic operators par document-level lock lagata hai, isliye chahe 1,000 log ek sath click karein, saveCount hamesha 100% accurate rehta hai.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Race condition, Atomic operators ($addToSet, $inc), Document-level locking, No read-modify-write flaw.

Project Code File: server/src/controllers/user.controller.ts (Line 35-52)

Q17. Agar Admin kisi shop ko delete kar de, toh uske reviews aur images ka kya hota hai?

Sir Ka Intention: Cascading deletion & Orphaned records question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, NoSQL me SQL jaisa automatic ON DELETE CASCADE nahi hota, isliye humne Application-Level Cascading Deletion implement kiya hai (admin.controller.ts -> deleteShop):
1. Reviews cleanup: await Review.deleteMany({ shopId });
2. Customer bookmarks cleanup: await User.updateMany({}, { $pull: { savedShops: shopId } });
3. Disk images cleanup: Node.js fs.unlink() se /uploads/ folder se shop ki saari cover photos delete hoti hain taaki server storage full na ho.
4. Audit log entry: Immutable Audit collection me record banta hai ki kis admin ne kab ye shop delete ki.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Application-level cascade, Orphaned data prevention, Disk file cleanup, Audit tracking.

Project Code File: server/src/controllers/admin.controller.ts -> deleteShop

TOPIC 7: VALIDATION, ERROR PIPELINE & ADVANCED FEATURES

Zod schema validation, asyncHandler wrapper, opening hours logic, aur audit logs.

Q18. Zod validation library kyu use ki? Joi ya manual if-else kyu nahi?

Sir Ka Intention: Modern validation pipeline question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, manual if-else me har route par 50 line ka code likhna padta hai jo maintain karna mushkil hota hai.
Zod TypeScript-first validation library hai:
1. Single Source of Truth: z.infer<typeof schema> use karke hum ek hi Zod schema se runtime validation bhi kar lete hain aur TypeScript ki static interface types bhi generate kar lete hain. Double coding nahi karni padti.
2. Strict Input Checking: Agar koi user ya attacker extra unexpected fields ya galat data type bhejta hai, toh Zod controller chalne se pehle hi 400 Bad Request throw kar deta hai clear field-level error messages ke sath.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Type inference (z.infer), Runtime schema validation, Zero redundant DTO interfaces.

Project Code File: server/src/middleware/validate.ts & server/src/schemas/

Q19. asyncHandler kya hota hai? Express 4 me async errors process crash kyu karte hain?

Sir Ka Intention: Express runtime deep question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, Express 4 ke synchronous route handlers me toh error catch ho jata hai, lekin async functions me agar await fail ho jaye aur try/catch na ho, toh Express use error-handling middleware tak nahi bhej pata. Isse unhandledRejection event trigger hota hai jo Node.js process ko crash kar deta hai!

Humne asyncHandler higher-order function banaya hai:
export const asyncHandler = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);

Ye har rejected promise ko automatically catch(next) karke global error handler tak pahunchata hai. Isse hume har controller me 10 baar repetitive try/catch nahi likhna padta.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Unhandled promise rejection, Higher-order wrapper, Centralized error pipeline, DRY principle.

Project Code File: server/src/utils/asyncHandler.ts

Q20. Server ko kaise pata chalta hai ki dukan 'Open Now' hai ya closed?

Sir Ka Intention: Live schedule calculation logic.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, Shop model me 7 dino ka schedule hota hai (Monday se Sunday open time aur close time jaise '09:00' se '22:00').
time.ts me isOpenNow() function chalta hai:
1. Current Indian Standard Time (IST) ka day (e.g. 'tuesday') nikalta hai.
2. Current time ko integer minutes me convert karta hai (jaise 14:30 = 14 * 60 + 30 = 870 minutes).
3. Shop ke open aur close time ko bhi minutes me convert karke compare karta hai (currentMinutes >= openMinutes && currentMinutes < closeMinutes).
4. Agar timing match kare toh openNow: true bhejta hai, varna reason bhejta hai (jaise 'Closed today' ya 'Opens tomorrow at 9 AM').

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Timezone normalization (Asia/Kolkata), Integer minute comparison, Live calculated projection.

Project Code File: server/src/utils/time.ts -> isOpenNow()

Q21. Shop views, call clicks aur direction clicks track karne se website slow kyu nahi hoti? (Fire-and-Forget)

Sir Ka Intention: Non-blocking asynchronous tracking question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, jab user kisi shop par click karta hai, agar hum page view record hone tak user ko rok ke rakhein, toh response 30ms slow ho jayega.

Isliye humne 'Fire-and-Forget' asynchronous pattern use kiya hai:

shop.controller.ts me hum shop ka data nikal kar turant ok(res, shop) return kar dete hain, aur analytics event ko void operator ke sath unawaited chhod dete hain:

void analyticsService.recordEvent({ shopId, type: 'view' });

User ko response turant deliver ho jata hai, aur Node.js ka event loop background me bina user ko roke database me analytics save kar deta hai.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: Fire-and-forget pattern, Non-blocking asynchronous I/O, Sub-50ms API response time.

Project Code File: server/src/controllers/shop.controller.ts (Line 20)

TOPIC 8: TESTING & VIVA PRESENTATION MASTER TIPS

150 Automated Tests ka proof aur viva me maximum marks laane ke 5 golden rules.

Q22. Aapne backend ko test kaise kiya? Kya automated tests likhe hain?

Sir Ka Intention: Testing rigor and quality assurance question.

Aapko Sir Ke Samne Ye Bolna Hai (Simple Hinglish):

Sir, humne Jest aur Supertest use karke 150 automated unit aur integration tests likhe hain jo 7 test suites me organized hain.

Test me real database corrupt na ho, iske liye humne 'mongodb-memory-server' use kiya hai jo RAM ke andar ek temporary isolated MongoDB instance banata hai.

Humne test kiya hai:

- auth.test.ts: Login, registration, token refresh, bcrypt encryption, wrong password checks.
- shops.test.ts: 2dsphere proximity search, category filters, owner edit gates.
- reviews-admin.test.ts: Average rating calculation, admin review deletion.
- regressions.test.ts: Redis fail hone par fallback resilience, simultaneous save race conditions.

Terminal me 'npm run test' chalane par saare 150 tests pass hote hain bina kisi failure ke.

Technical Keywords Jo Examiner Ko Impress Karenge:

Keywords to speak: 150 automated tests, Jest & Supertest, mongodb-memory-server, 100% test pass rate.

`Project Code File: server/package.json ('test': 'jest --runInBand')`

KAL VIVA ME FULL MARKS SCORE KARNE KE 5 GOLDEN RULES:

1. Confidence ke sath shuru karo: Examiner se seedhe bolo: 'Sir, hamara backend sirf basic CRUD nahi hai. Isme humne MongoDB 2dsphere Geospatial engine, 5-stage Price Comparison aggregation pipeline, Redis caching, aur Dual-Token JWT lagaya hai.'
2. Examiner ko Code File dikhao: Jab sir puchein ki 'Nearby search kahan likha hai?', toh turant server/src/models/Shop.ts khol kar line dikhao jahan '2dsphere' likha hai. Code dikhane wale student ko examiner turant maximum marks deta hai.
3. 150 Automated Tests ka zikr zaroor karo: College projects me 95% students tests nahi likhte. Jab aap bologe ki 'Sir humne Jest se 150 automated integration tests likhe hain jo pass hote hain', toh examiner bahut impress hoga.
4. Edge Cases par ghabrao mat: Agar sir puchein 'Agar internet chala jaye toh kya hoga?', toh bolo: 'Sir Winston logger error record karta hai aur next phase me hum RabbitMQ se offline queues implement karenge.'
5. Smile aur Positive Body Language rakho: Examiner aapko confuse karne ki koshish karega, lekin jo answer is PDF me diya hai uspe firm raho.